MINISTÉRIO DA DEFESA • EXÉRCITO BRASILEIRO • IME

Curso de Graduação em Engenharia da Computação

Projeto de Final de Curso • 2026

Planejamento de Trajetória para Robô Terrestre usando Grafos

Apresentação final: da imagem aérea ao UGV em simulação

Autores

Gabriel Dias Vilela • **Tassandro Cavalcante Leitão**

Orientadores

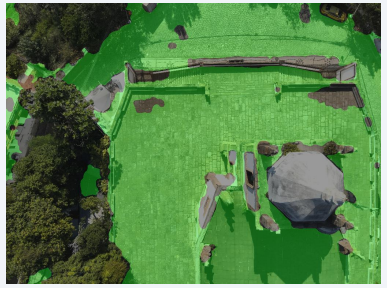
Profa. Claudia Marcela Justel, D.Sc. • Prof. Paulo Fernando Ferreira Rosa, Ph.D.

Rio de Janeiro • 2026

Contexto e motivação

Por que planejar trajetórias de UGV a partir de imagens aéreas

- **Emprego militar e de resgate:** robôs autônomos aumentam a eficácia operacional e reduzem a exposição e o desgaste humano em combate e em eventos extremos (terremotos, chuvas intensas, acidentes radioativos)
- **Planejamento de trajetória:** tarefa essencial à autonomia de robôs terrestres; o cenário é modelado como um grafo, com vértices nas posições transitáveis e arestas definindo os caminhos
- **Pesquisa de referência (LIARC):** cooperação entre veículos não tripulados aéreos (UAV) e terrestres (UGV), iniciada em 2023 pelo Maj Fábio Luiz Junior
- **Aplicação potencial:** projeto estratégico Sistemas de Veículos Terrestres Remotamente Pilotados (SVTRP), do CTEx
- **Um caminho ótimo sobre uma imagem não garante que um veículo consiga segui-lo:** a simulação permite essa verificação sem risco e sem plataforma física



1. O UAV fotografa a área e a segmentação separa o que é transitável



2. O mapa é convertido em grafo e os algoritmos calculam o caminho



3. O UGV percorre a trajetória (Clearpath Jackal no Gazebo)

Ponto de partida: o PFC 2025

Almeida e Barbosa (2025): planejamento por otimização em grafos sobre imagens já segmentadas

Principais conclusões do PFC 2025

- Dijkstra e A*: desempenho mais consistente, confiável e previsível que o RRT (determinismo e otimalidade)
- A* ligeiramente mais rápido que o Dijkstra, explorando menos vértices: mais adequado ao problema
- Pesos nas arestas por proximidade a obstáculos: caminhos mais realistas, sem impacto significativo no tempo
- RRT com dificuldade em áreas transitáveis estreitas: excedeu o tempo limite em 2 de 5 cenários
- Segmentação + planejamento em grafos: alternativa promissora para veículos autônomos

Frentes de continuidade

Ambientes dinâmicos

apontada pelo PFC 2025

Grafos em composição de imagens em tempo real

apontada pelo PFC 2025

Segmentação semântica para mais tipos de terreno

apontada pelo PFC 2025 → foco deste projeto

Execução efetiva do caminho por um veículo (simulado)

nova frente, não prevista no PFC 2025

Objetivos

Objetivo principal e objetivos específicos, com o grau de cumprimento

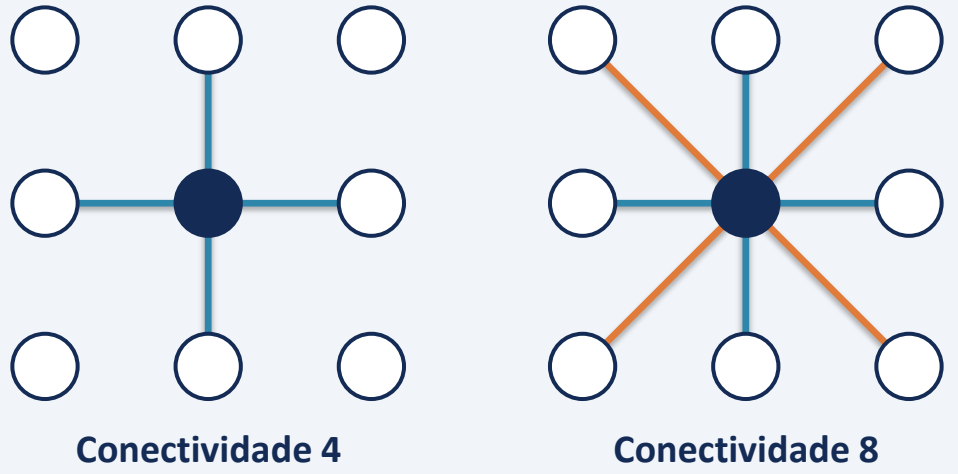
Objetivo principal: construir um sistema modular capaz de, a partir de uma imagem aérea real, identificar as áreas transitáveis, planejar uma trajetória por diferentes algoritmos e executar essa trajetória em um veículo terrestre, validando-a previamente em ambiente simulado.

- ✓ **Arquitetura orientada a objetos com cinco módulos encadeados**
  Realizado
- ✓ **RRT sob a mesma interface, comparado a BFS, DFS, Dijkstra e A***
  Realizado
- ✓ **Segmentação semântica com os modelos pré-treinados (Junior, 2024b)**
  Realizado
- ✓ **Ponte ROS 2 e UGV simulado (Clearpath Jackal) no Gazebo**
  Realizado
- ✓ **Caracterizar os percursos e a correspondência métrica imagem–mundo**
  Realizado
- ✗ **Validar o sistema em ambiente real, com veículo físico**
  Não realizado: plataforma, espaço e tempo

Do mapa ao grafo

Planejamento global: a imagem aérea descreve por completo o ambiente de operação do UGV

- **Planejamento global:** todo o ambiente é conhecido antes de o robô partir; no planejamento local, o ambiente é descoberto durante o trajeto
- **Vértices:** cada pixel classificado como transitável
- **Arestas:** ligam pixels vizinhos; quais vizinhos contam é definido pela conectividade
- **Conectividade 8 adotada:** permite movimentos diagonais e caminhos mais retos, com ~2× mais arestas; a conectividade 4 gera caminhos em escada
- **Grafo direcionado:** as arestas (u, v) e (v, u) são guardadas separadamente, pois o peso depende do vértice de chegada
- **Tamanho do problema:** uma imagem de 1000 × 750 px gera centenas de milhares de vértices e milhões de arestas; construir o grafo rápido importa tanto quanto buscar rápido



círculo = pixel transitável; azul: vizinhos laterais; laranja: diagonais

Custo assimétrico: ir de uma região livre para perto de um obstáculo custa mais do que o movimento inverso; por isso cada ligação entre vizinhos é contada como duas arestas, uma em cada sentido.

Algoritmos de busca em grafos

Os quatro algoritmos clássicos implementados, e o papel de cada um neste projeto

BFS

O(|V| + |E|)

Busca em largura: minimiza o número de arestas. Tende ao caminho geometricamente mais curto, mas ignora a penalização e passa rente aos obstáculos.

DFS

não garante o menor caminho

Busca em profundidade: segue um ramo até o fim antes de voltar. Serve só como base de comparação; em grade, percorre quase toda a área transitável.

Dijkstra

O((|V| + |E|) log |V|)

Encontra o caminho de menor custo (pesos não negativos), examinando os vértices em ordem crescente de custo, em todas as direções.

A*

ótimo se h não superestimar

Como o Dijkstra, mas guiado por uma estimativa da distância até o destino (distância em linha reta). Por isso explora menos vértices.

$$f(n) = g(n) + h(n)$$

Menos vértices ≠ menos tempo: para cada vizinho, o A* ainda calcula a estimativa (uma raiz quadrada). Em grafos grandes e regulares, como os gerados a partir de imagens, esse trabalho extra pode anular o ganho.

Árvore de Exploração Rápida Aleatória (RRT)

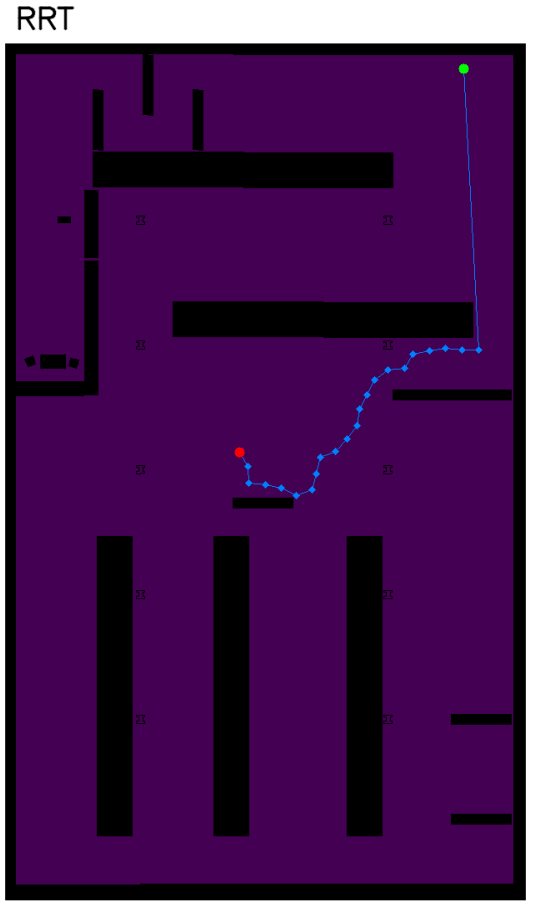
Planejador por sorteio de pontos (LaValle, 1998): uma árvore cresce a partir da partida, explorando o espaço livre

1. Sorteia um ponto qualquer do mapa
2. Encontra o vértice da árvore mais próximo desse ponto
3. Avança, a partir dele, um passo de tamanho fixo em direção ao ponto
4. Se o trecho não atravessa obstáculos, adiciona o novo vértice

Parada: quando um novo vértice alcança o destino em linha reta, sem obstáculos. **Para acelerar:** em parte dos sorteios, usa-se o próprio destino.

Vantagens: cada iteração é barata; se existe caminho, acaba encontrando; sorteios podem ser repetidos

Limitações: caminho não é o melhor e sai irregular; tem dificuldade em passagens estreitas



RRT no mapa do galpão simulado: vértices da árvore em círculos (detalhe da Figura 7)

Comparação justa?

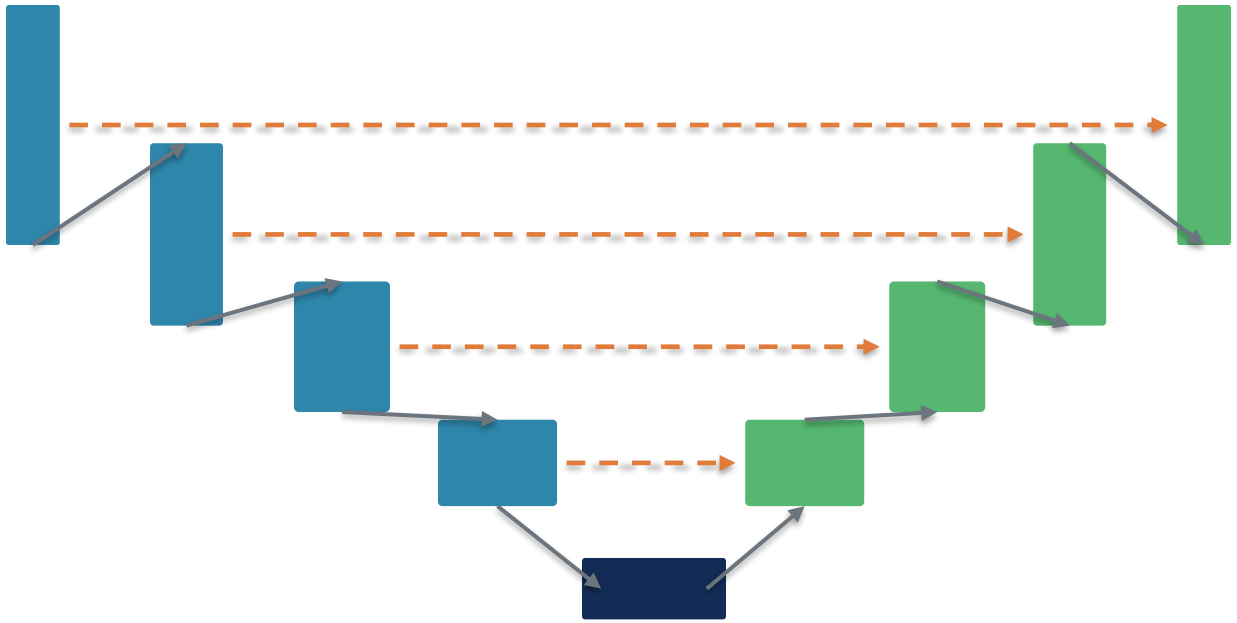
O RRT não usa as arestas do grafo: seu custo é só o comprimento do caminho, sem as penalidades dos demais.

Obstáculos móveis

Rápido e sem precisar refazer o grafo a cada mudança: bom candidato para replanejar (trabalho futuro).

Arquitetura U-Net e mapa de navegabilidade

Ronneberger, Fischer e Brox (2015): rede que classifica cada pixel da imagem



Codificador
reduz a imagem e reconhece o conteúdo

Atalhos
devolvem os detalhes de posição

Decodificador
volta ao tamanho original

Modelos já treinados (Junior, 2024b) no Semantic Drone Dataset (imagens aéreas de 5 a 30 m, 23 classes): o sistema só os executa, no lugar da separação por cor (HSV) do PFC 2025. Codificadores: ResNet 50/101, EfficientNet-B4/B5 e MobileNet V2.

Mapa de navegabilidade

Formato definido em (Junior, 2025): as regiões transitáveis são pintadas com duas cores fixas, BGR (84, 1, 68) e (0, 0, 255).

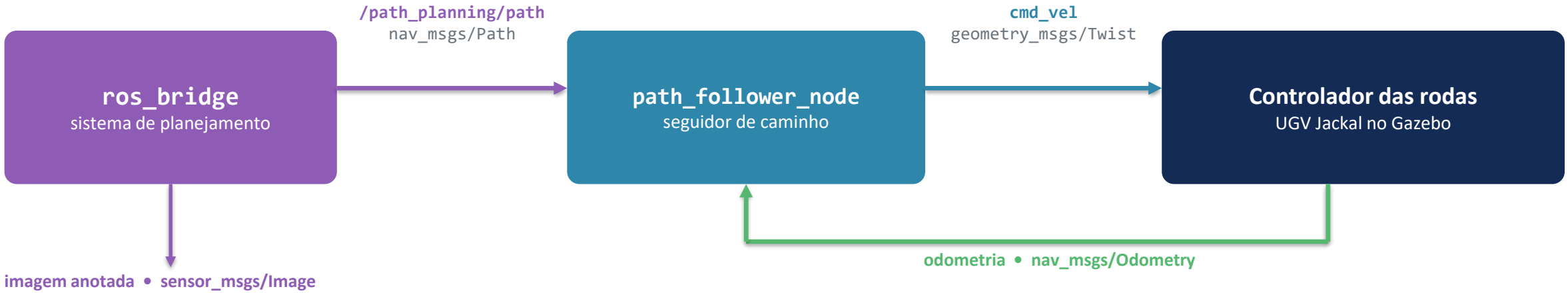
BGR (84, 1, 68)

BGR (0, 0, 255)

Vantagem: o mapa pode vir de qualquer fonte (rede neural, separação por cor ou mundo simulado); basta usar essas cores. Para saber se há obstáculo entre dois pontos, verificam-se as cores ao longo da reta.

ROS 2 e simulação no Gazebo

Programas independentes (nós) trocam mensagens por canais nomeados (tópicos)



- **Nó:** programa com uma função (ler sensor, planejar, comandar motores); não é o mesmo que vértice de grafo
- **Publicar e assinar:** quem envia não precisa saber quem recebe; assim, o robô simulado pode ser trocado por um real sem mudar o planejamento
- **Gazebo:** simulador de código aberto; mundos e robôs descritos em arquivos SDF (posições e formas dos objetos)

Entrega de mensagens a quem chega depois

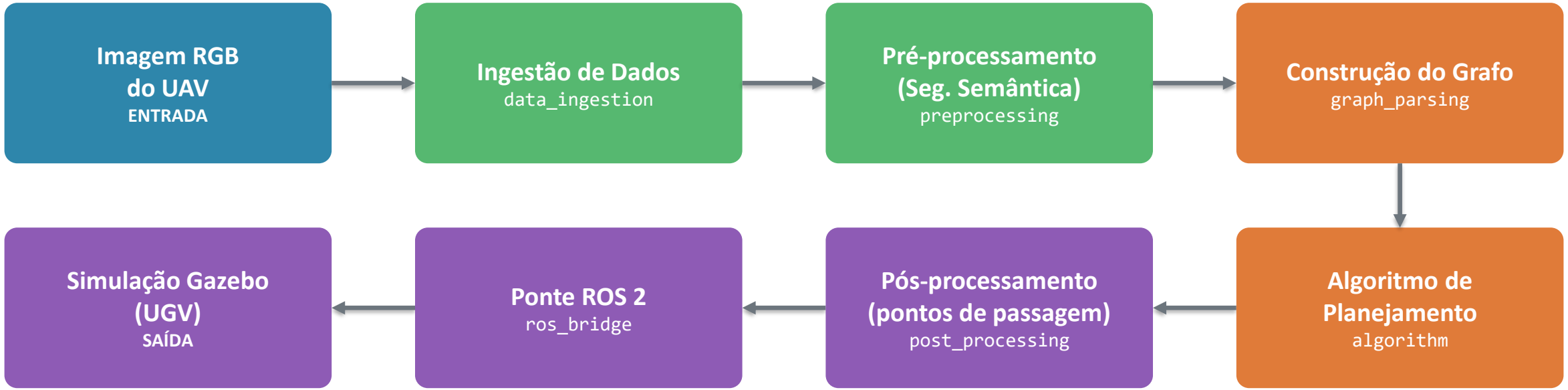
volatile: só recebe quem já estava assinando o tópico.

transient local: guarda a última mensagem e a reenvia a quem assinar depois.

O planejador publica o caminho uma única vez e encerra; com transient local, o robô o recebe assim que os dois nós se encontram na rede.

Arquitetura do sistema

Cinco módulos independentes, em sequência; cada um pode ser trocado sem mexer nos outros



`IngestedData` → `ImageIngestedData` organiza os dados recebidos do UAV

`SegmentedImageData` máscara de navegabilidade

`Graph` atributos dos vértices, lista de arestas e atributos das arestas

`PathResult` vértices, custo total e metadados

`RobotPath` pontos de passagem, imagens do caminho e envio ao ROS 2

Cada módulo: interfaces.py, entities.py, index.py (fábrica de objetos) e __init__.py. Para trocar modelo, algoritmo ou saída, basta registrar uma nova classe.

Função de peso e custo das arestas

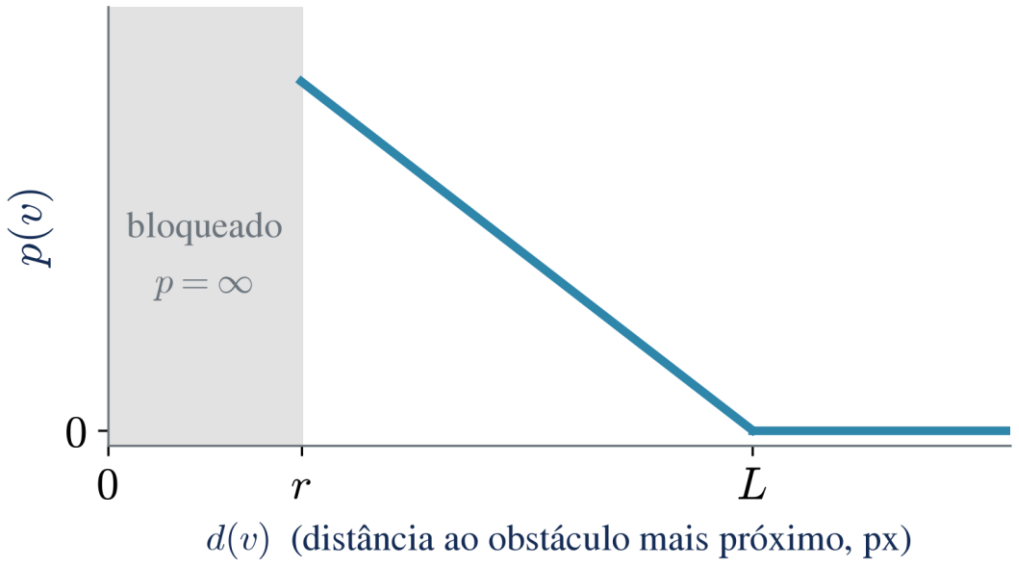
Penalizar trajetórias próximas a obstáculos, sem proibi-las

$$p(v) = \begin{cases} \infty, & \text{se } d(v) \le r \quad \text{(vértice bloqueado)} \\ L - d(v), & \text{se } r < d(v) < L \\ 0, & \text{se } d(v) \ge L \end{cases}$$

d(v): distância do pixel ao obstáculo mais próximo; r (robot_size): folga mínima, igual ao raio do robô mais uma margem; L (weight_limit): distância a partir da qual o obstáculo deixa de pesar.

$$w(e) = \delta(e) + p(v), \qquad \delta(e) \in \{1, \sqrt{2}\}$$

- Dijkstra e A* usam w(e); δ(e) é o comprimento do passo, usado para medir a distância (em metros, × m/px)
- A penalidade depende do vértice de chegada: w(u, v) ≠ w(v, u), por isso o grafo é direcionado
- O custo ℓ(P) não é uma distância: soma pixels e penalidades



Forma da penalidade p(v) em função de d(v) (ilustração da definição)

O parâmetro L define o quanto o planejador prefere passar pelo meio dos corredores; a mais de L pixels, o obstáculo não influencia mais.

Lei de controle do seguidor de caminho

path_follower_node: a cada ciclo, compara a posição estimada do robô com o próximo ponto de passagem (x_t, y_t)

Erro de orientação: quanto o robô precisa girar, entre −π e π

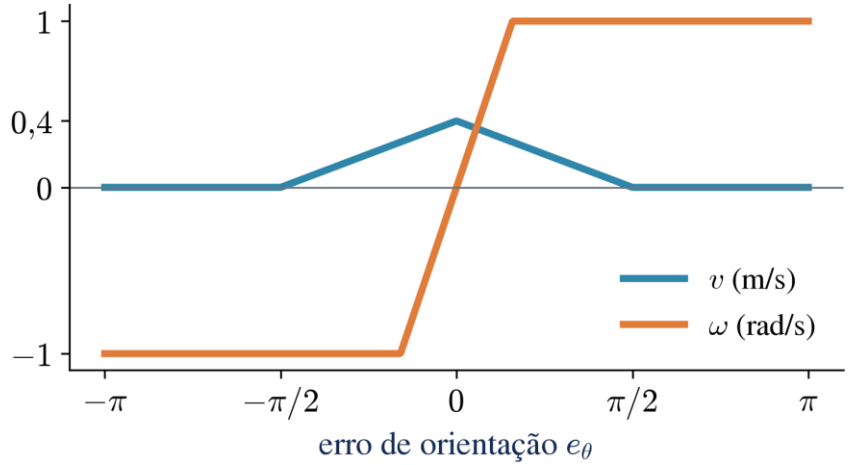
$$e_\theta = \operatorname{atan2}(\sin(\theta_d - \theta),\ \cos(\theta_d - \theta)), \quad \theta_d = \operatorname{atan2}(y_t - y,\ x_t - x)$$

O uso de seno e cosseno garante que o robô gire sempre pelo lado mais curto.

Comandos de velocidade (geometry_msgs/Twist)

$$\omega = \operatorname{sat}(2\, e_\theta,\ \omega_{\max}), \qquad v = v_{\text{cruzeiro}} \cdot \max\left(0,\ 1 - \frac{|e_\theta|}{\pi/2}\right)$$

A velocidade de giro ω cresce com o erro, até um limite; a de avanço v cai com o desalinhamento e zera acima de 90°: o robô não anda na direção errada enquanto se alinha.



v e ω em função do erro de orientação, com os parâmetros adotados

0,4 m/s	1,0 rad/s	0,12 m	0,15 m
velocidade de cruzeiro	giro máximo	tolerância em cada ponto	tolerância na chegada

Um ponto de passagem é considerado alcançado quando o robô chega a menos da tolerância dele.

Mapa de ocupação a partir do mundo simulado

Script gazebo_world_to_occupancy.py: transforma o mundo do Gazebo em um mapa que o sistema entende

Problema: a imagem e o mundo simulado eram lugares diferentes; escala e origem eram ajustadas à mão, e o robô fazia o desenho certo, porém deslocado ou no tamanho errado.

1. **Leitura do mundo**
   lê todos os objetos do arquivo, inclusive os importados de outras pastas e do repositório de modelos

2. **Posição de cada objeto**
   combina as posições das partes de cada objeto para obter onde ele está no mundo

3. **Desenho visto de cima**
   caixas, cilindros e malhas 3D projetados no chão; pisos e tetos descartados; objetos não encontrados são ignorados, com aviso

4. **Geração do mapa**
   área livre na cor transitável e obstáculos em preto; informa a resolução (m/px) e a origem (m) exatas

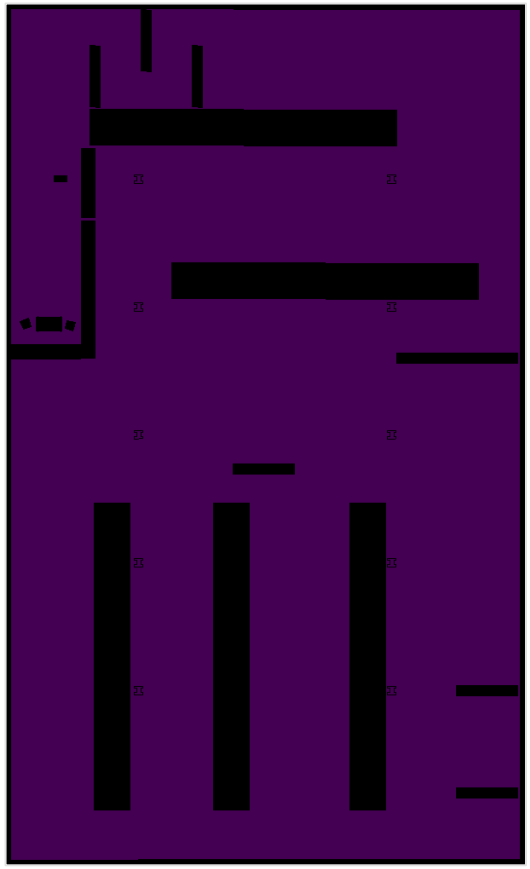


Figura 4: mapa do galpão simulado (mundo warehouse). Roxo: área livre; preto: obstáculos.

~30 × 50 m
galpão fechado

0,05 m/px
resolução

863 × 1242
pixels

Resultado: erros do planejador não se confundem mais com erros de alinhamento entre imagem e mundo.

Comparação dos modelos de segmentação

Imagem aérea do Outeiro da Glória reduzida para 1024 × 768 px, processada na GPU integrada

Modelo	mIoU	Acurácia	Área transit. (%)	Tempo (s)
ResNet 50	0,3933	0,8244	65,5	0,72
ResNet 101	0,4032	0,8720	67,5	0,52
EfficientNet-B4	0,5012	0,9120	61,1	0,71
EfficientNet-B5	0,4819	0,9025	69,4	0,57

Tabela 2: mIoU e acurácia divulgadas em (Junior, 2024b); área transitável e tempo medidos neste trabalho.

- **EfficientNet-B4:** maior mIoU (0,5012) e o mais cauteloso: marca só 61,1% da imagem como transitável, com bordas mais bem delimitadas
- **Todos abaixo de 1 s:** o tempo quase não muda entre modelos, então a mIoU decide a escolha
- **De 61,1% a 69,4%:** o modelo muda a área livre entregue ao planejador (marcar demais pode incluir riscos; marcar de menos pode fechar passagens)

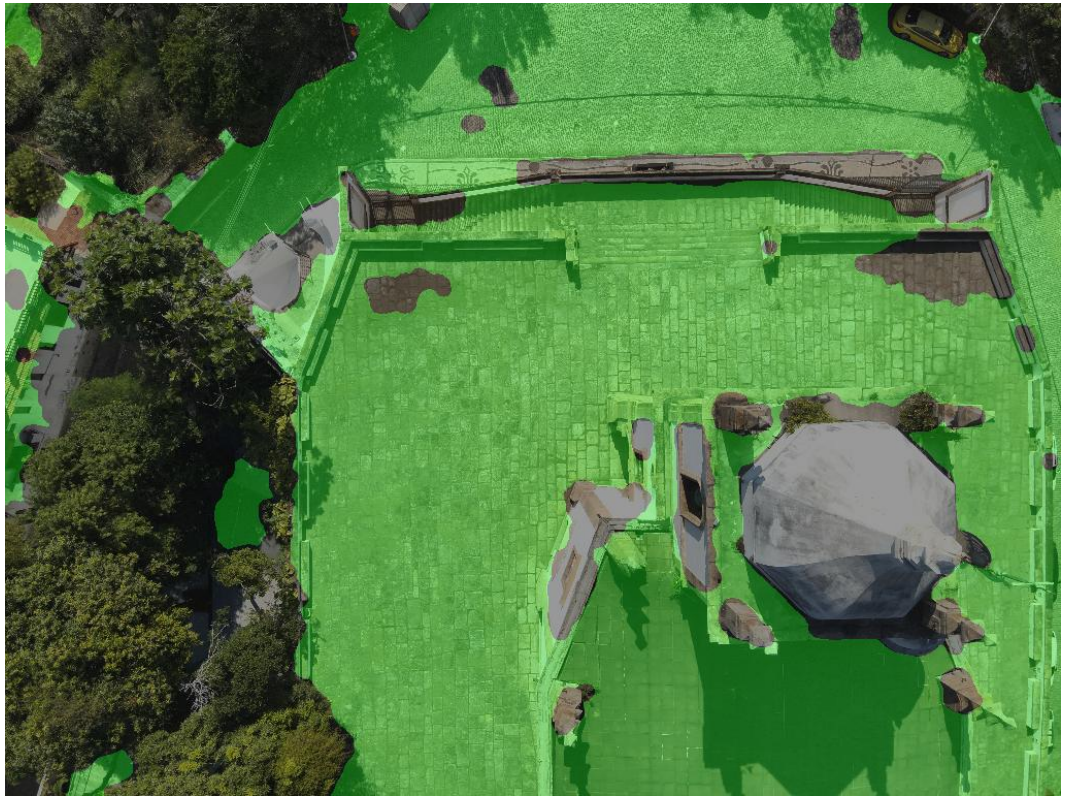


Figura 5: EfficientNet-B4; em verde, a região classificada como transitável. Pátio e vegetação rasteira incluídos; igreja, edificações, árvores densas e veículos excluídos, inclusive em regiões sombreadas.

Algoritmos sobre imagem aérea segmentada

Outeiro da Glória, 1000 × 750 px: ~440 mil vértices e 3,3 milhões de arestas; mesma partida e chegada para todos

Algoritmo	Vértices (k+1)	Custo ℓ(P)	Vért. expl. / iter.	Tempo (s)	Distância (px)
BFS	351	12.838,8	179.546	0,131	398,0
DFS	65.282	1.314.372,7	248.479	0,159	92.241,7
Dijkstra	879	1.462,2	228.525	0,392	990,7
A*	879	1.462,2	203.472	0,426	990,7
RRT	30	931,6*	289	0,010	931,8

Tabela 3. * Para o RRT, o custo é o comprimento geométrico, não comparável ao custo ponderado dos demais.

- **A* e Dijkstra:** mesmo custo mínimo (1.462,2); o A* explora ~11% menos vértices, mas não foi mais rápido
- **BFS:** menor distância (398,0 px), mas custo ponderado muito maior: passa rente aos obstáculos
- **DFS:** 65.282 vértices, distância ~100× a do A*; RRT: o mais rápido (0,010 s, 289 iterações)



Figura 6: amarelo = não transitável; partida em vermelho, chegada em verde. DFS omitida (encobriria os demais).

Algoritmos sobre o mapa do galpão simulado

Mundo warehouse, 863 × 1242 px, 0,05 m/px: 502.833 vértices e 3.753.214 arestas; tempo = mediana de 3 execuções

Algoritmo	Vértices (k+1)	Custo ℓ(P)	Vért. expl. / iter.	Tempo (s)	Distância (px)
A* (ponderado)	609	28.714,2	391.239	2,173	709,1
Dijkstra (ponderado)	609	28.714,2	397.659	1,694	709,1
BFS	599	44.174,4	414.703	0,674	722,3
DFS	2.614	233.982,4	467.072	0,843	3.693,7
RRT	23	758,5*	230	0,073	757,3

Tabela 4. * Comprimento do caminho do RRT (sorteios repetidos: as 3 execuções deram o mesmo caminho).

- **A* e Dijkstra:** caminhos ligeiramente diferentes, mesmo custo (28.714,2), pelo meio dos corredores
- **BFS:** custo 54% maior, com trajeto colado às bordas; DFS: distância ~5× a do A*
- **RRT:** 23 vértices, 757,3 px contra 709,1 px, com mudanças bruscas de direção

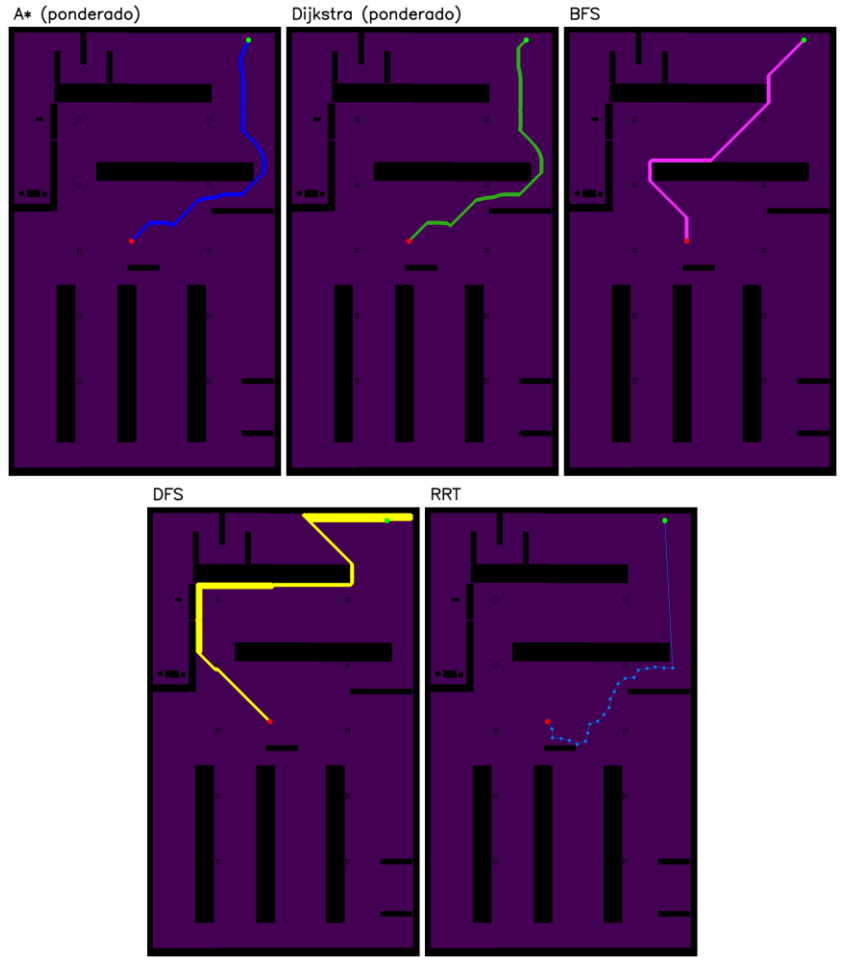
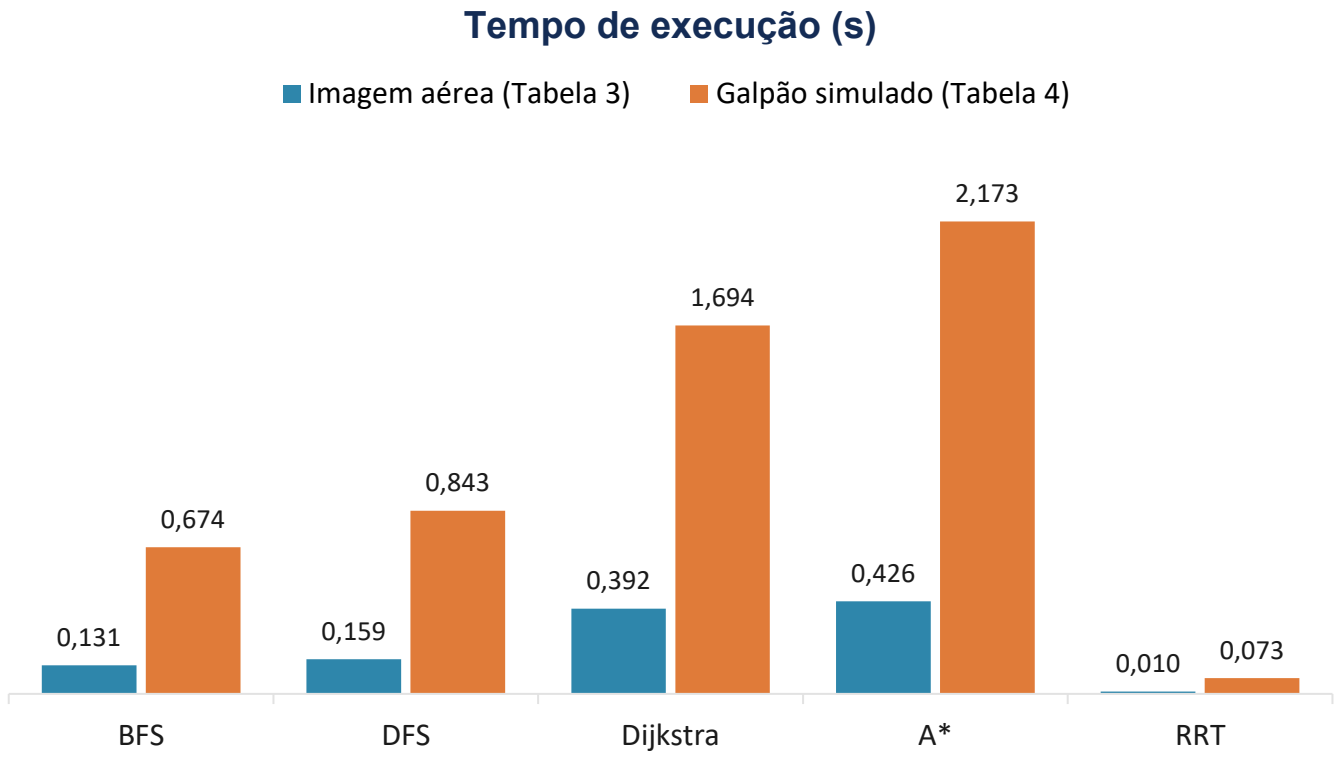


Figura 7: caminhos desenhados separadamente (recorte da área interna do galpão).

Busca em grafos × RRT: tempo e forma do caminho

Tempos de planejamento das Tabelas 3 e 4



9 a 43× mais rápido: tempo de planejamento do RRT comparado ao dos algoritmos de busca, nos dois cenários

Por que não adotar o RRT?

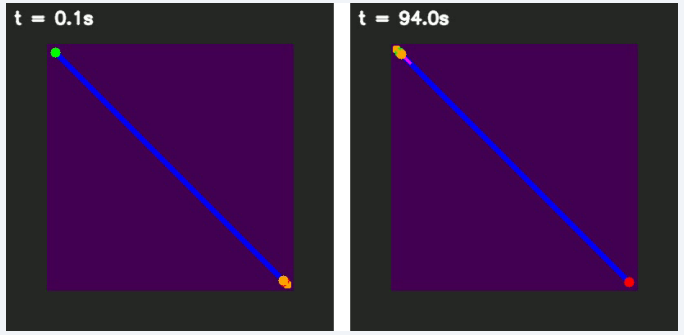
- Busca em grafos: curvas suaves, que acompanham os corredores
- RRT: trechos curtos com mudanças bruscas de direção, sem relação com os obstáculos, e um trecho final longo
- Depende do cenário: quase reto na área aberta; cheio de desvios no galpão
- Cada mudança brusca obriga o robô a frear e girar no lugar

Decisão: apesar de o RRT planejar muito mais rápido, os ensaios no simulador usaram o A* com pesos.

Validação no simulador Gazebo

Teste de ponta a ponta: o UGV percorre o caminho recebido e para ao chegar ao destino

✓ robô alcançou o destino



1 Mapa artificial

Teste básico: uma reta diagonal sem obstáculos. Confere envio do caminho, seguimento e parada no destino.

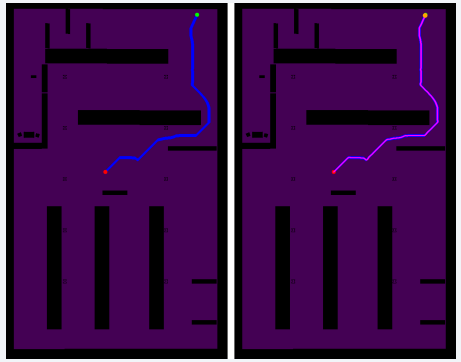
✓ robô alcançou o destino



2 Imagem aérea real

Caminho planejado sobre a imagem do Outeiro da Glória segmentada pela U-Net (MobileNet V2).

✓ robô alcançou o destino



3 Galpão simulado

Mapa tirado do próprio mundo simulado: único ensaio em que o caminho desvia dos obstáculos que o robô encontra.

Em todos: galpão simulado, simulador sem interface gráfica, caminho do A* com pesos enviado pela ponte ROS 2, velocidade de cruzeiro de 0,4 m/s; o rastro do robô é desenhado a partir da odometria.

Ensaio 2: percurso sobre imagem aérea real

Outeiro da Glória, reduzida para 800 × 600 px e segmentada pela U-Net com MobileNet V2

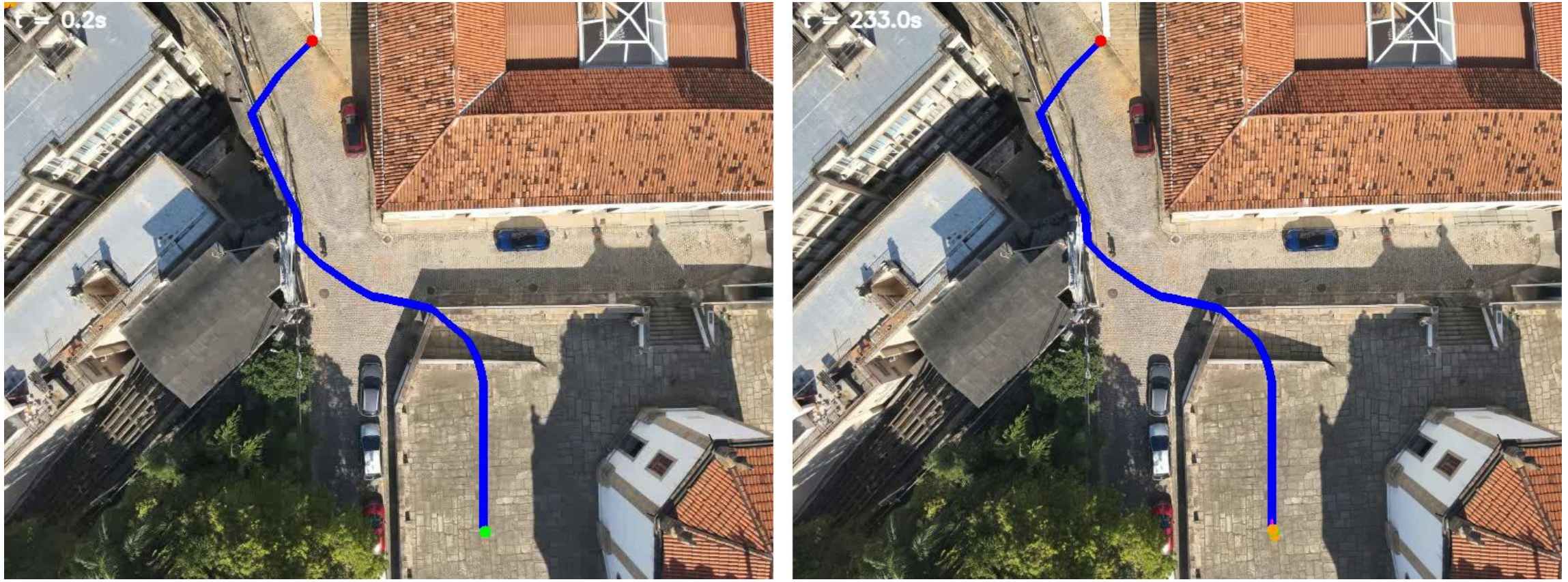


Figura 9: (a) entrada: caminho do A* (azul), o único enviado, partida em vermelho e chegada em verde; (b) resultado (t = 233,1 s): UGV simulado (laranja) na chegada e rastro recente em magenta.

Da rua estreita (alto) ao pátio pavimentado (embaixo), contornando edificações e veículos. Registro de 232,9 s, incluindo o trajeto inicial até o primeiro ponto; testa se o robô segue o caminho, e não se desvia dos obstáculos do mundo simulado.

Ensaio 3: galpão simulado, com o mapa do próprio mundo

Mesmos pontos da Tabela 4, com folga mínima de 12 px (0,6 m) pelas dimensões do Jackal

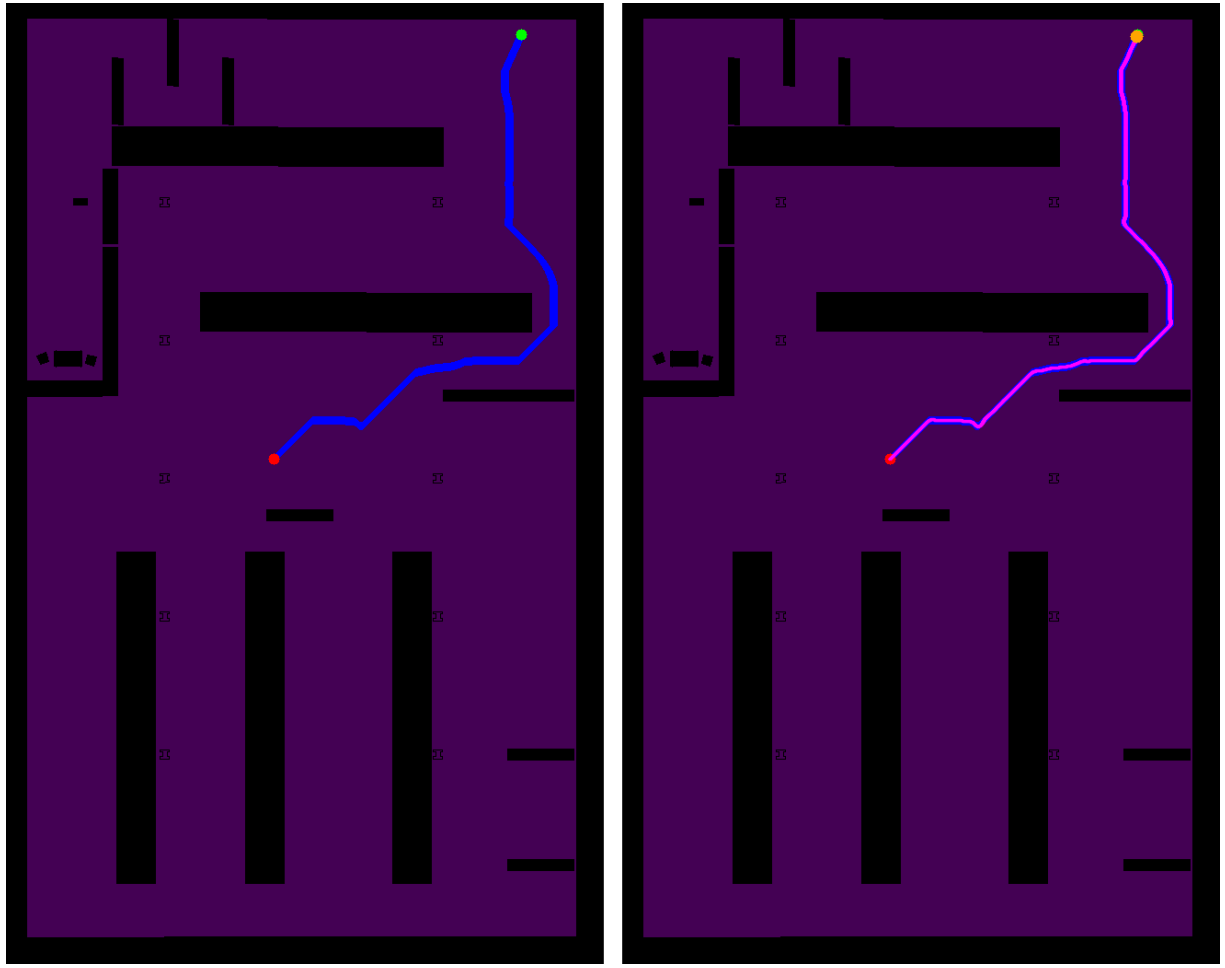


Figura 10: (a) caminho planejado pelo A*; (b) rastro completo da odometria (magenta) e posição final (laranja).

621 pontos de passagem	**35,8 m** caminho planejado
35,1 m percorridos	**95,8 s** de movimento
0,37 m/s velocidade média	**4.537** amostras (~47/s)

Único ensaio em que o mapa corresponde exatamente ao mundo: o caminho desvia dos obstáculos que o robô encontra. O robô partiu do primeiro ponto do caminho; velocidade média próxima à de cruzeiro (0,4 m/s).

Quanto o robô se afastou do caminho planejado

Erro lateral: para cada posição registrada, a menor distância até o caminho planejado

0,8 cm
erro lateral médio

1,3 cm
raiz do erro quadrático médio

2,9 cm
95% das medidas abaixo disso

7,1 cm
erro máximo

Todos bem abaixo da tolerância dos pontos (0,12 m). O robô parou a 0,15 m da chegada, exatamente a tolerância de chegada configurada.

Onde estão os desvios: nas mudanças de direção: o caminho planejado faz um canto vivo, e o robô faz uma curva.

Limitação da medida: a posição vem da odometria, a mesma estimativa que o robô usa para se guiar. O número mostra quão bem ele segue o caminho, e não quão bem sabe onde está; num robô real, os erros da odometria se acumulariam. Nos ensaios 1 e 2 os dados não foram salvos: avaliação só visual.

Limitação: ausência de validação em ambiente real

Prevista para os dois últimos meses do cronograma, a etapa não foi realizada

1 Plataforma

Exigiria um UGV com odometria confiável, unidade inercial e interface ROS 2 disponível para uso continuado no período de ensaios, o que não pôde ser assegurado no prazo.

2 Espaço e imagem

Espaço aberto com obstáculos permanentes e percursos de dezenas de metros, e voo de UAV com autorização, meteorologia adequada e medição de referência da escala; eventual repetição do voo.

3 Tempo

A integração com o ROS 2 e o simulador tomou boa parte do prazo: comunicação entre nós, configuração do simulador e, principalmente, falhas da interface gráfica.

O que foi verificado (em malha fechada)

Envio do caminho, seguimento dos pontos de passagem, correção pela odometria e parada no destino, com as mesmas interfaces de um robô real.

O que só o mundo real mostraria

Terreno irregular e escorregadio, erros de odometria acumulados em percursos longos, erros de posicionamento da imagem e obstáculos que não aparecem na foto.

Conclusões: o sistema

Do PFC 2025, que planejava sobre imagens já segmentadas, à execução do caminho por um veículo simulado

Arquitetura modular
Cinco módulos independentes: novos algoritmos ou modelos entram sem alterar os demais.

RRT incluído
RRT e busca em grafos comparados diretamente, sobre os mesmos mapas.

Segmentação integrada
U-Net já treinada: o sistema parte de fotos reais de drone, e não de mapas já prontos.

ROS 2 + Gazebo
O caminho calculado passou a ser percorrido por um UGV simulado.

Escala e posição corretas
Mapa gerado a partir do próprio mundo simulado: sem ajuste manual de escala e origem.

3 de 3

cenários simulados concluídos, com o veículo chegando ao destino em todos

< 1 cm

de distância média entre o caminho percorrido e o planejado, no galpão simulado

Perspectivas futuras

Em ordem aproximada de prioridade

1

Validação em ambiente real

UGV físico, foto aérea da área de teste e escala medida em campo, para enfrentar terreno irregular e erros de odometria

2

Erro lateral para todos os algoritmos

Medir o afastamento do robô também nos caminhos dos outros algoritmos: qual é mais fácil de seguir

3

Ambientes dinâmicos

Replanejamento em tempo real com RRT e variantes, ou replanejamento incremental em grafos (1ª perspectiva do PFC 2025)

4

RRT com várias execuções

Repetir com sorteios diferentes e medir média e variação de comprimento, tempo e mudanças de direção

5

Suavização do caminho

Trocar cantos vivos por curvas que o robô consiga fazer; ajuda sobretudo o RRT e a BFS

6

Menos memória no grafo

Calcular arestas e pesos só quando a busca precisar deles, em vez de guardar todos

Em resumo: da imagem aérea ao UGV em simulação

Arquitetura modular
5 módulos; algoritmos e modelos podem ser trocados

Segmentação semântica
U-Net identifica as áreas transitáveis

Busca em grafos + RRT
A* com pesos escolhido para guiar o robô

ROS 2 + Gazebo
3 percursos concluídos, desvio médio de 0,8 cm

Próximo passo: validação em ambiente real, sem mudanças no software.

Obrigado.

Gabriel Dias Vilela • Tassandro Cavalcante Leitão